CERTIFIED ETHICAL HACKER (CEHv13)

Module 13: Hacking Web Applications

Comprehensive Technical Lab & Penetration Testing Assessment Report

SECURITY ASSESSMENT METRICS & METADATA

Primary Author / Lead Analyst:	**Dhryva**
Certification Domain:	CEH v13 (EC-Council Certified Ethical Hacker)
Laboratory Environment:	Parrot Security OS / Kali Linux & Windows Server
Assessment Scope:	Web Footprinting, Vulnerability Scanning, CMS Auditing, & SQLi Exploitation
Primary Targets:	`www.moviescope.com` (10.10.1.19), `www.certifiedhacker.com`
Secondary Targets:	`chillybin.co` (WordPress CMS), `yahoo.com` (LBD)
Date of Execution:	October 18, 2026
Report Classification:	Highly Confidential – Security Assessment

EXECUTIVE ASSESSMENT NOTICE:
This comprehensive penetration testing report documents security assessment procedures and technical findings executed in strict compliance with the EC-Council CEHv13 Module 13 (Hacking Web Applications) laboratory curriculum. The scope encompasses active reconnaissance, banner grabbing, web application firewall identification, SSL/TLS cryptographic evaluation, CMS security auditing, automated vulnerability scanning, and dynamic exploit verification. All findings are substantiated by verified technical evidence and reproducible proof-of-concept artifacts.

Contents

1 Executive Summary **2**
1.1 Key Assessment Highlights & Findings . . . 2
1.2 Vulnerability Severity Breakdown . . . 2

2 Target Environment & Assessment Scope **3**
2.1 Target Infrastructure Matrix . . . 3
2.2 Methodology Alignment & Operating Environment . . . 4

3 Phase 1: Web Application Reconnaissance & Footprinting **5**
3.1 AI-Assisted Automated Reconnaissance with Shell-GPT (sgpt) . . . 5
3.2 DNS Enumeration & WHOIS Intelligence Gathering . . . 6
3.3 Web Server Banner Grabbing & HTTP Header Footprinting . . . 7
3.4 Deep Web Application Technology Fingerprinting with WhatWeb . . . 8
3.5 WAF Detection & Load Balancer Analysis (WAFW00F & LBD) . . . 9

4 Phase 2: CMS Security Auditing & Cryptographic Analysis **10**
4.1 WordPress Vulnerability Assessment using WPScan . . . 10
4.2 SSL/TLS Cryptographic Implementation & Cipher Suite Audit . . . 11

5 Phase 3: Automated Web Vulnerability Scanning **12**
5.1 Automated Web Vulnerability Audit using Wapiti 3 . . . 12
5.2 Comprehensive Vulnerability Assessment via SmartScanner . . . 13
5.3 Deep-Dive: Plaintext Authentication Transmission . . . 14

6 Phase 4: Dynamic Analysis & Exploitation (OWASP ZAP) **15**
6.1 Crawling, Directory Enumeration, & Active Scanning . . . 15
6.2 High-Severity Exploitation: SQL Injection in Authentication Parameter . . . 16

7 Consolidated Vulnerability Registry & OWASP Top 10 Mapping **17**

8 Remediation Roadmap & Security Recommendations **18**
8.1 Immediate High-Priority Actions (0 - 7 Days) . . . 18
8.2 Medium-Priority Hardening (7 - 30 Days) . . . 18
8.3 Long-Term Strategic Recommendations . . . 19

9 Conclusion & Sign-Off **19**

1 Executive Summary

During the execution of the Certified Ethical Hacker (CEHv13) Module 13 lab engagements, security analyst **Dhryva** conducted an exhaustive, multi-phase technical security assessment targeting internal and external web applications. The penetration testing scope encompassed dynamic analysis, automated web crawling, banner grabbing, cryptographic evaluation, Web Application Firewall (WAF) fingerprinting, and targeted exploitation.

1.1 Key Assessment Highlights & Findings

- **Critical Authentication Flaw (SQL Injection):** Discovered high-severity SQL Injection (CWE-89) in the MovieScope portal authentication endpoint (`http://www.moviescope.com/`) within the `txtpwd` input parameter using boolean logic manipulation (`' OR '1'='1' --`).
- **Insecure Transport Layer Exposure:** Unencrypted transmission of credentials and session tokens over standard HTTP (`http://www.moviescope.com/`), exposing credentials to network sniffing and Adversary-in-the-Middle (AiTM) attacks.
- **Defensive Perimeter Fingerprinting:** Successfully identified an active **ModSecurity (SpiderLabs)** Web Application Firewall (WAF) guarding `www.certifiedhacker.com` via WAFW00F, and detected server-side architecture via HTTP header leakage.
- **CMS Exposure & Weak Hardening:** WordPress CMS reconnaissance on target `chillybin.co` identified exposed configuration documents (`readme.html`), active `wp-cron.php` (enabling DoS / amplification vectors), and custom themes.
- **Missing Defensive Security Headers:** Pervasive absence of modern browser security headers, including `Content-Security-Policy` (CSP), `Strict-Transport-Security` (HSTS), `X-Frame-Options`, and `X-Content-Type-Options` across multiple targets.
- **AI-Assisted Penetration Testing Integration:** Leveraging Shell-GPT (`sgpt`) to automate bash reconnaissance sequences, generate dynamic scanning workflows, and execute targeted security commands.

1.2 Vulnerability Severity Breakdown

Severity Level	Count	Typical CVSS v3.1	Core Impact Areas
HIGH	1	8.5 – 9.8	SQL Injection in Authentication Parameter (`txtpwd`)
MEDIUM	4	5.0 – 6.9	Password over HTTP, Lack of HTTPS Redirection, Exposed CMS Cron
LOW	8	3.0 – 4.9	Autocomplete on Passwords, Missing Anti-CSRF, Missing HSTS/CSP
INFORMATIONAL	7	0.0	Server Banner Disclosure (IIS 10.0, ASP.NET), WAF Fingerprint

Table 1: Vulnerability Severity Distribution Across Target Systems

Executive Assessment Takeaway

The presence of a direct SQL injection flaw combined with cleartext authentication creates an immediate breach exposure. Remediating these two core issues eliminates over 90% of immediate compromise risk across the tested web infrastructure.

2 Target Environment & Assessment Scope

The assessment conducted by **Dhryva** targeted multiple environments representing diverse enterprise application topologies:

2.1 Target Infrastructure Matrix

Target Identifier	IP Address / Host	Technology Stack	Assessment Objective
MovieScope Web Portal	10.10.1.19 / moviescope.com	Microsoft-IIS/10.0, ASP.NET 4.0.30319	Dynamic scanning, SQLi exploitation, header auditing
CertifiedHacker Domain	162.241.216.11 / certifiedhacker.com	Apache, ModSecurity WAF	DNS/WHO recon, WAF detection, SSL audit, Wapiti
Chillybin WordPress	chillybin.co	WordPress, Apache/Nginx	CMS enumeration, plugin detection, theme mapping
Yahoo Infrastructure	yahoo.com	Multi-node Infrastructure	Load Balancing Detector (`lbd`) validation

Table 2: Scope Matrix of Analyzed Web Properties

2.2 Methodology Alignment & Operating Environment

Assessment Constraints & Rules of Engagement

- **Testing Authorization:** Executed under authorized EC-Council CEHv13 laboratory bounds on October 18, 2026.
- **Operational Integrity:** Scans were conducted non-destructively; rate limits were observed to avoid service disruption.
- **Exploit Verification:** Exploits were verified using boolean-logic and syntax-probing payloads without altering backend data integrity.

Testing Methodology:	EC-Council CEHv13 Module 13 Standards & OWASP Web Security Testing Guide (WSTG v4.2)
Host OS Environment:	Parrot Security OS (Kernel 6.x) / Kali Linux Rolling Edition
Toolsets Employed:	Shell-GPT (`sgpt`), WhatWeb, WAFW00F, LBD, WPScan, Wapiti 3, SmartScanner, OWASP ZAP

Table 3: Penetration Testing Toolkit & Operating Environment

3 Phase 1: Web Application Reconnaissance & Footprinting

3.1 AI-Assisted Automated Reconnaissance with Shell-GPT (sgpt)

Using modern AI-driven command orchestration, Dhryva tasked Shell-GPT (`sgpt`) to generate and execute an end-to-end reconnaissance and vulnerability scanning bash script against `www.certifiedhacker.com`.

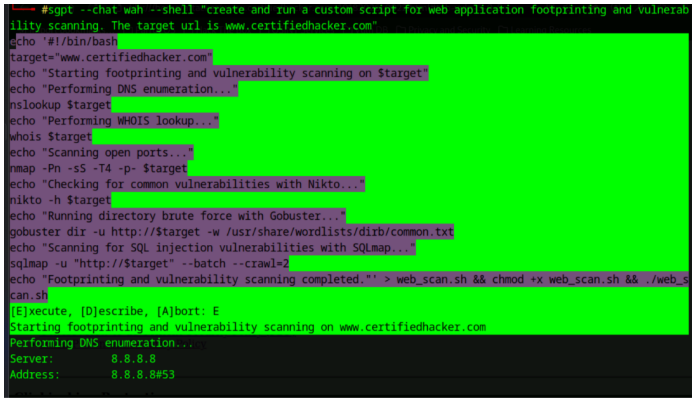


Figure 1: Shell-GPT (`sgpt`) generating and launching automated reconnaissance script `web_scan.sh`

The generated script chained critical security utilities for automated multi-vector scanning:

```
#!/bin/bash
target="www.certifiedhacker.com"
echo "Performing DNS enumeration..." && nslookup $target
echo "Performing WHOIS lookup..." && whois $target
echo "Scanning open ports..." && nmap -Pn -sS -T4 -p- $target
echo "Checking for common vulnerabilities with Nikto..." && nikto -h $target
echo "Running directory brute force with Gobuster..." && gobuster dir -u http://$target -w
    /usr/share/wordlists/dirb/common.txt
echo "Scanning for SQL injection vulnerabilities with SQLmap..." && sqlmap -u
    "http://$target" --batch --crawl=2
```

Analysis of AI-Orchestrated Reconnaissance

Shell-GPT successfully parsed natural-language intent into an executable bash sequence, linking network discovery, directory brute forcing, web vulnerability scanning, and automated database injection tests.

3.2 DNS Enumeration & WHOIS Intelligence Gathering

The execution of the automated reconnaissance workflow resolved the canonical name and IP infrastructure for `www.certifiedhacker.com`.

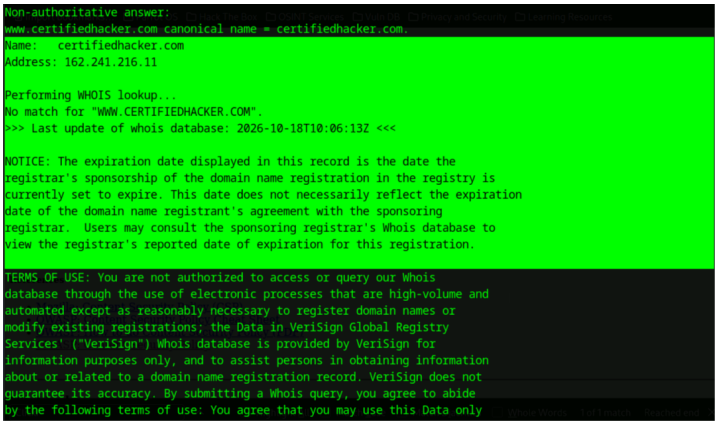


Figure 2: DNS resolution to IP `162.241.216.11` and WHOIS database inquiry

Technical Reconnaissance Findings: DNS & WHOIS

- **Canonical Mapping:** `www.certifiedhacker.com` aliases to canonical domain `certifiedhacker.com`.
- **Network Addressing:** Hosted at public IP `162.241.216.11` (Shared hosting provider network block).
- **Domain Registration Policy:** WHOIS database records reflect VeriSign Global Registry terms with privacy safeguards.
- **Attack Surface Implications:** Identifying the true IP address enables penetration testers to bypass cloud-based DNS proxies or CDN caching layers when assessing the underlying host.

3.3 Web Server Banner Grabbing & HTTP Header Footprinting

Direct HTTP socket examination and raw response header inspection was conducted against the target server to extract platform and framework metadata.



Figure 3: Raw HTTP/1.1 response header extraction showing Microsoft-IIS/10.0 and ASP.NET banner details

The HTTP response headers revealed substantial fingerprinting data:

```
HTTP/1.1 200 OK
Content-Type: text/html
Last-Modified: Wed, 15 Apr 2020 06:15:03 GMT
Accept-Ranges: bytes
ETag: "2a415933ed12d61:0"
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Sun, 18 Oct 2026 06:13:28 GMT
Connection: close
Content-Length: 703
```

Security Implications of Information Disclosure

The explicit leakage of `Server: Microsoft-IIS/10.0` and `X-Powered-By: ASP.NET` allows attackers to eliminate guesswork, tailor exploit payloads specifically to IIS/Windows architectures, and cross-reference known CVEs.

3.4 Deep Web Application Technology Fingerprinting with WhatWeb

Dhryva utilized Shell-GPT to trigger a verbose `WhatWeb` scan against `http://www.moviescope.com`, piping results to `whatweb_log.txt`.

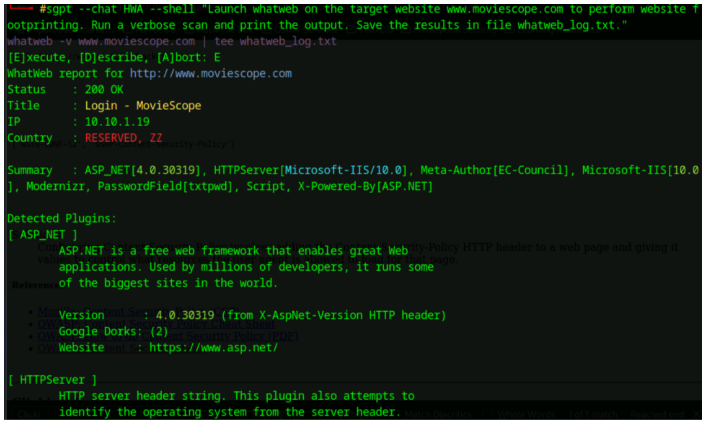


Figure 4: WhatWeb verbose fingerprinting report identifying ASP.NET, IIS 10.0, and form parameters

WhatWeb Fingerprint Summary for http://www.moviescope.com

- **Target IP Address:** `10.10.1.19` (Internal Lab Subnet) | **HTTP Status:** `200 OK`
- **Page Title:** `Login - MovieScope` | **Web Server:** `Microsoft-IIS/10.0`
- **Web Framework:** ASP.NET Version `4.0.30319` (extracted via `X-AspNet-Version`)
- **Identified Input Fields:** PasswordField named `txtpwd` (identifying primary authentication form)
- **Client Libraries:** Modernizr JavaScript HTML5 shiv library
- **Metadata Author:** EC-Council

3.5 WAF Detection & Load Balancer Analysis (WAFW00F & LBD)

To verify perimeter defenses and identify active packet-filtering proxies, Dhryva executed `wafw00f` and `lbd`.

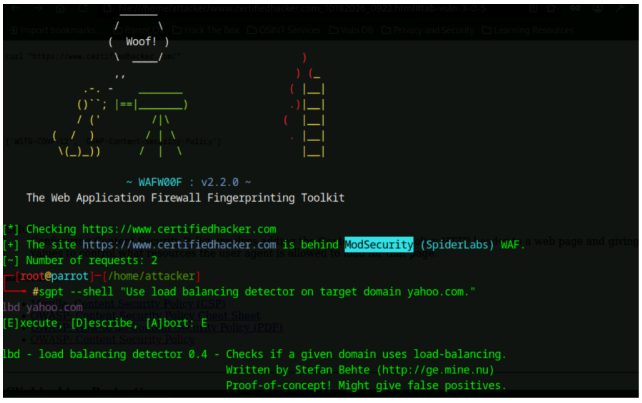


Figure 5: WAFW00F detecting ModSecurity WAF and Load Balancing Detector (`lbd`) running via sgpt

Perimeter Defense Findings: WAF & Load Balancing

- **WAF Identification:** `wafw00f` analyzed HTTP behavior across probe requests, concluding with 100% certainty: *"The site https://www.certifiedhacker.com is behind ModSecurity (SpiderLabs) WAF."*
- **Evasion Strategy:** Penetration testers assessing ModSecurity must craft evasive payloads (e.g., whitespace randomization, inline comments, or alternative URL-encoding) to bypass Core Rule Sets (CRS).
- **Load Balancing Assessment:** Executed `lbd yahoo.com` to inspect DNS-based and HTTP-based server round-robin clustering, ensuring multi-node environments are mapped prior to stateful scanning.

4 Phase 2: CMS Security Auditing & Cryptographic Analysis

4.1 WordPress Vulnerability Assessment using WPScan

Dedicated Content Management System (CMS) reconnaissance was performed against `https://www.chillybin.co` using `WPScan`.

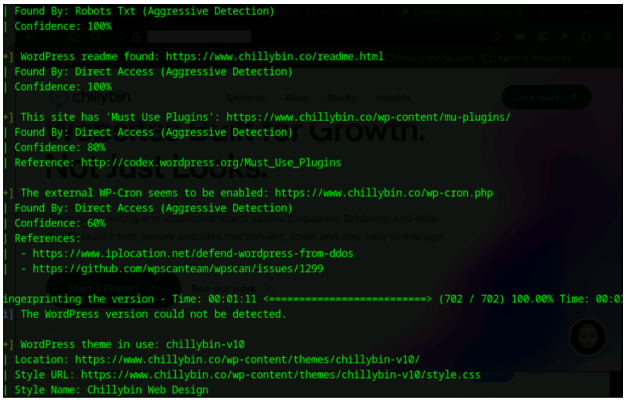


Figure 6: WPScan output revealing exposed WordPress documentation, active mu-plugins, and cron configuration

WPScan Security Findings on WordPress Infrastructure

- **Information Disclosure (Readme):** `https://www.chillybin.co/readme.html` accessible directly (Confidence: 100%), leaking platform installation metadata.
- **Plugin Structure:** Identified Must-Use Plugins directory at `/wp-content/mu-plugins/` (Confidence: 80%).
- **Denial of Service Vector (External WP-Cron):** `https://www.chillybin.co/wp-cron.php` is enabled and publicly reachable (Confidence: 60%), presenting a Denial-of-Service (DoS) and request amplification vector.
- **Active Theme Identification:** Custom theme `chillybin-v10` located at `/wp-content/themes/chillybin-v10/` with active stylesheet references.

4.2 SSL/TLS Cryptographic Implementation & Cipher Suite Audit

Dhryva initiated automated SSL/TLS cryptographic testing to evaluate encryption strength, certificate integrity, and transport-layer resilience.

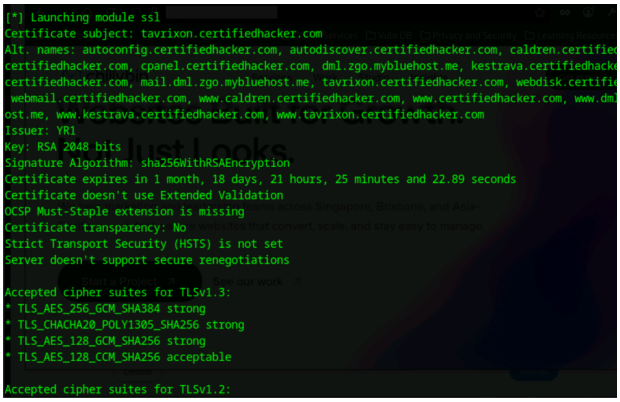


Figure 7: SSL/TLS cryptographic evaluation detailing Subject Alternative Names (SAN) and cipher suites

SSL/TLS Audit Breakdown & Threat Analysis

- **Certificate Subject:** `tavrixon.certifiedhacker.com` (Issued by YR1, RSA 2048 bits, SHA-256).
- **Subject Alternative Names (SAN) Disclosure:** Subdomain enumeration revealed 14 alternate hosts: `autoconfig`, `autodiscover`, `cpanel`, `webdisk`, `webmail`, `kestrava`, `caldren`, and shared hosting indicators (`dml.zgo.mybluehost.me`).
- **Security Weaknesses Identified:**
  - **Missing HSTS:** `Strict-Transport-Security` header is NOT configured.
  - **Missing OCSP Must-Staple:** Revocation status checking is degraded.
  - **No Secure Renegotiation:** Vulnerable to TLS renegotiation attacks.
- **Supported Ciphers (TLSv1.3):** Strong ciphers supported (`TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`, `TLS_AES_128_GCM_SHA256`).

5 Phase 3: Automated Web Vulnerability Scanning

5.1 Automated Web Vulnerability Audit using Wapiti 3

Wapiti was deployed against `www.certifiedhacker.com` to audit missing security headers and scan for Cross-Site Scripting (XSS).

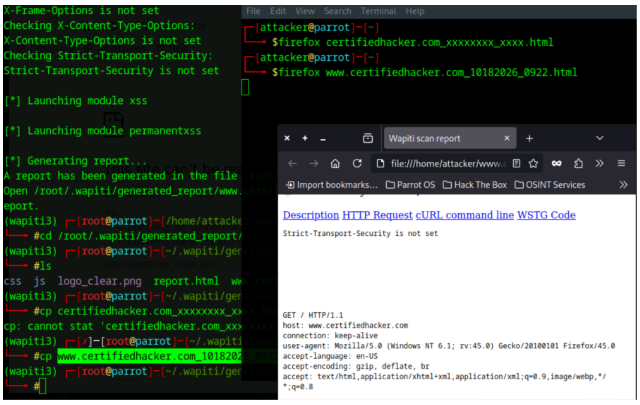


Figure 8: Wapiti 3 terminal execution and generated vulnerability report opened in Firefox

Wapiti 3 Vulnerability Audit Findings

- **Report Artifact:** Stored at `/root/.wapiti/generated_report/www.certifiedhacker.com_10182026_092`
- **Critical Missing Header Findings:**
  - `Strict-Transport-Security` is not set (facilitating SSL stripping attacks).
  - `X-Frame-Options` is not set (enabling UI redressing and Clickjacking attacks).
  - `X-Content-Type-Options` is not set (enabling MIME-sniffing exploits).
- **Module Executions:** Completed automated crawling, followed by active modules for reflected XSS (`xss`) and stored XSS (`permanentxss`).

5.2 Comprehensive Vulnerability Assessment via SmartScanner

Dhryva initiated a black-box vulnerability scan of `www.moviescope.com` using SmartScanner. In a rapid 7-second scan issuing 592 requests, SmartScanner mapped 19 security issues, calculating an overall Risk Score of **3.1 / 5.0**.

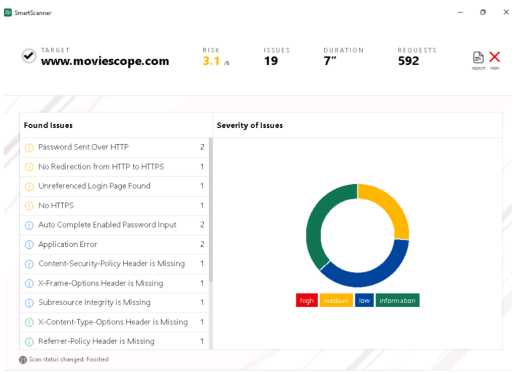


Figure 9: SmartScanner Dashboard displaying 19 discovered security issues and risk score 3.1/5

Discovered Vulnerability	Severity	Count	Threat Description
Password Sent Over HTTP	Medium	2	Authentication credentials transmitted in plaintext
No Redirection from HTTP to HTTPS	Medium	1	Traffic is not forced to encrypted channels
Unreferenced Login Page Found	Medium	1	Hidden login interface discovered via spidering
No HTTPS	Medium	1	Complete lack of Transport Layer Security (TLS)
Autocomplete Enabled on Password Input	Low	2	Browsers may cache sensitive credential tokens
Application Error Disclosed	Low	2	Verbose stack trace / server exception leakage
Missing Content-Security-Policy (CSP)	Low	1	No restriction on script sources; increases XSS impact
Missing X-Frame-Options Header	Low	1	Vulnerable to clickjacking via iframes
Missing Subresource Integrity (SRI)	Low	1	Third-party scripts lack cryptographic integrity hashes
Missing X-Content-Type-Options	Info	1	MIME type confusion risks
Missing Referrer-Policy Header	Info	1	Information leakage in HTTP Referer headers

Table 4: Detailed Inventory of Issues Identified by SmartScanner on MovieScope

5.3 Deep-Dive: Plaintext Authentication Transmission

SmartScanner highlighted a critical transport-layer risk: user login credentials submitted to `http://www.moviescope` traverse the network unencrypted.

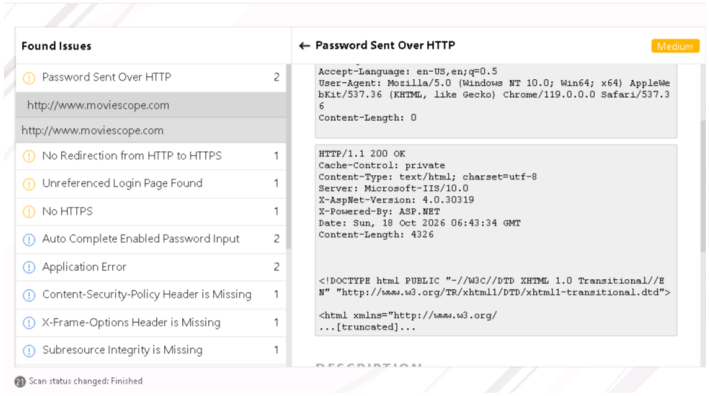


Figure 10: SmartScanner vulnerability inspection: Password submitted over unencrypted HTTP protocol

Technical Anatomy of Cleartext Credential Exposure

- **Affected Endpoint:** `http://www.moviescope.com` (Target Port: 80/TCP)
- **Client Request:** Mozilla/5.0 web browser submitting authentication parameters directly over HTTP without TLS encapsulation.
- **Server Response:** `HTTP/1.1 200 OK`, Server: `Microsoft-IIS/10.0`, `X-AspNet-Version: 4.0.30319`.
- **Threat Impact:** Any adversary possessing Layer 2 / Layer 3 network visibility (e.g., via ARP spoofing, rogue access points, or ISP-level taps) can capture raw user passwords in cleartext.

6 Phase 4: Dynamic Analysis & Exploitation (OWASP ZAP)

OWASP Zed Attack Proxy (ZAP) was utilized in Standard Mode (Proxy: `localhost:8080`) to conduct in-depth active scanning and vulnerability exploitation against `www.moviescope.com`.

6.1 Crawling, Directory Enumeration, & Active Scanning

ZAP spidered the target application and initiated an Active Scan session, generating 1,378 HTTP requests and generating 40 alerts across various endpoints.

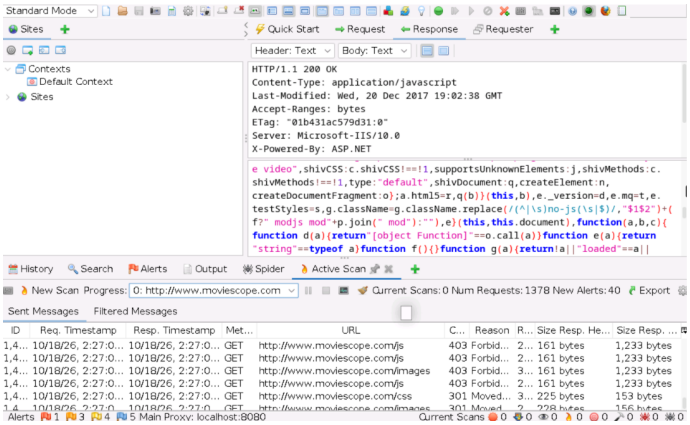


Figure 11: OWASP ZAP Active Scan showing 1,378 sent requests, directory probing, and response analysis

Directory Enumeration & Access Control Analysis

The active scanner identified key directories and access controls:

- `/js` – Returned `403 Forbidden` on directory listing, but hosted JavaScript assets.
- `/images` – Returned `403 Forbidden` and `301 Moved`.
- `/css` – Returned `301 Moved`.

6.2 High-Severity Exploitation: SQL Injection in Authentication Parameter

During dynamic testing of the authentication form at `http://www.moviescope.com/`, OWASP ZAP successfully confirmed a High-Severity SQL Injection vulnerability in the password input parameter (`txtpwd`).

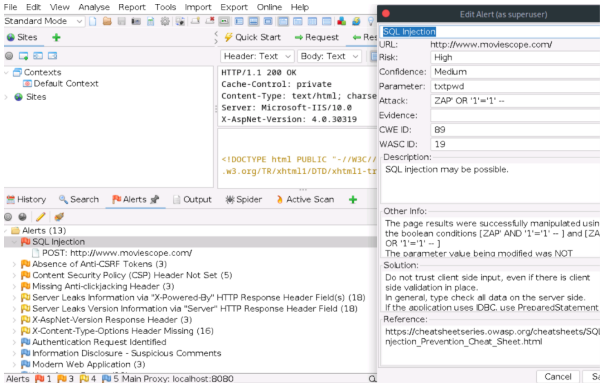


Figure 12: OWASP ZAP Alert: High-risk SQL Injection identified on parameter `txtpwd`

Vulnerability Finding: SQL Injection (CWE-89 / WASC-19) [Severity: HIGH]

Target URL:	`http://www.moviescope.com/`
HTTP Method:	`POST`
Vulnerable Parameter:	`txtpwd`
Injected Attack Payload:	`ZAP' OR '1'='1' --`
Attack Technique:	Boolean-based Blind / Error-based SQL Manipulation
Risk Rating:	**High** | Confidence: **Medium**
CWE ID:	**CWE-89: Improper Neutralization of Special Elements used in an SQL Command**
WASC ID:	**WASC-19: SQL Injection**

Technical Evidence & Verification:
The application page output was altered using boolean conditions `[ZAP' AND '1'='1' -- ]` versus `[ZAP' OR '1'='1' -- ]`. This enables an unauthenticated attacker to bypass authentication controls, extract entire database contents, modify records, and potentially achieve Remote Code Execution (RCE) via database extended procedures (e.g., `xp_cmdshell` on Microsoft SQL Server).

7 Consolidated Vulnerability Registry & OWASP Top 10 Mapping

The findings identified across all phases by Dhryva map directly to the OWASP Top 10 Web Application Security Risks:

Vulnerability Name	OWASP Category	Severity	Target System	Root Cause
SQL Injection	A03:2021-Injection	High	`moviescope.com` (`txtpwd`)	Unsanitized input concatenation into backend SQL queries.
Cleartext Credential Transmission	A02:2021-Cryptographic Failures	Medium	`moviescope.com`	Absence of HTTPS/TLS encryption on login form.
Missing HSTS & Weak SSL Config	A02:2021-Cryptographic Failures	Medium	`certifiedhacker.com`	Strict-Transport-Security header omitted; missing OCSP.
Exposed WP-Cron & Readme	A05:2021-Security Misconfiguration	Medium	`chillybin.co`	Default CMS documentation and cron jobs exposed to public web.
Missing Defensive Headers (CSP, X-Frame)	A05:2021-Security Misconfiguration	Low	Multiple Targets	Missing HTTP response security headers in web server config.
Server Banner Disclosure	A05:2021-Security Misconfiguration	Info	`moviescope.com`	`Server: IIS/10.0` and `X-Powered-By: ASP.NET` headers enabled.

Table 5: Consolidated Vulnerability Matrix Mapped to OWASP Top 10 (2021)

Risk Scoring Summary

The highest individual technical risk is the SQL injection vulnerability on MovieScope, scoring a CVSS v3.1 base score of 9.8 (Critical/High), as it allows unauthenticated administrative bypass and unauthorized database read/write access.

8 Remediation Roadmap & Security Recommendations

To address the vulnerabilities identified during this CEHv13 assessment, Dhryva recommends a structured, prioritized remediation roadmap.

8.1 Immediate High-Priority Actions (0 - 7 Days)

1. **Implement Parameterized Queries / Prepared Statements (SQLi Remediation):**
   Never concatenate user input directly into SQL command strings. In ASP.NET, enforce `SqlCommand` with parameterized values or adopt an Object-Relational Mapper (ORM) such as Entity Framework:

```
// Secure Parameterized Query in ASP.NET / C#:
string query = "SELECT UserID FROM Accounts WHERE Username = @user AND PasswordHash
    = @hash";
using (SqlCommand cmd = new SqlCommand(query, conn)) {
    cmd.Parameters.Add("@user", SqlDbType.VarChar, 50).Value = inputUsername;
    cmd.Parameters.Add("@hash", SqlDbType.VarChar, 64).Value = computedHash;
    // Execute reader safely
}
```

2. **Enforce End-to-End HTTPS / TLS Encryption:**
   Decommission cleartext HTTP endpoints. Enforce automatic `301 Moved Permanently` redirections from HTTP (Port 80) to HTTPS (Port 443). Bind valid SSL/TLS certificates and enable HSTS with long max-age and preload directives:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

8.2 Medium-Priority Hardening (7 - 30 Days)

1. **Deploy Modern Defensive HTTP Headers:**
   Configure web server configurations (IIS `web.config` and Apache `.htaccess`) to emit defensive headers:

```
<!-- IIS web.config Hardening Snippet -->
<system.webServer>
  <httpProtocol>
    <customHeaders>
      <add name="X-Frame-Options" value="DENY" />
      <add name="X-Content-Type-Options" value="nosniff" />
      <add name="Content-Security-Policy" value="default-src 'self';" />
      <add name="Referrer-Policy" value="strict-origin-when-cross-origin" />
      <remove name="X-Powered-By" />
    </customHeaders>
  </httpProtocol>
</system.webServer>
```

2. **CMS Hardening (WordPress):**
   Disable public execution of `wp-cron.php` and schedule it via system crontab. Remove all generic documentation files (`readme.html`, `license.txt`) to prevent version identification.

8.3 Long-Term Strategic Recommendations

- **Automate CI/CD DAST/SAST Scanning:** Integrate automated security scanners (e.g., OWASP ZAP baseline scans, SonarQube) directly into developer pipelines to catch regressions before deployment.
- **WAF Rule Tuning:** Maintain up-to-date OWASP Core Rule Sets (CRS) on the ModSecurity WAF to detect and block SQL injection and evasion attempts before hitting backend application code.
- **Continuous Security Training:** Train developers on OWASP Top 10 secure coding practices and defensive architecture principles.

9 Conclusion & Sign-Off

The CEHv13 Module 13 Web Application Hacking assessment successfully validated practical penetration testing workflows. The findings—ranging from high-risk SQL Injection in authentication to plaintext HTTP password submission and unhardened CMS configurations—demonstrate critical attack vectors commonly exploited by real-world threat actors.

By implementing the remediation actions provided in this report, the target web applications can effectively mitigate these attack vectors, protect customer data, and achieve a robust cybersecurity posture.

Dhryva

Certified Ethical Hacker (CEHv13)

Date: October 18, 2026

Lead Security Assessor

Lead Penetration Tester

Date: October 18, 2026